Query Processing in B$^+$-Trees

1 Query Processing in a B$^+$-Tree

Query processing in a B$^+$-Tree leverages its balanced, multi-level structure and leaf-node chaining. We discuss two primary types: **point queries** and **range queries**.

1.1 Point Query

A *point query* retrieves the record with a specific search-key value v. The search always starts at the root and follows pointers down the tree until the target leaf is reached.

Algorithm 1 Find(v) – Point Query in a B$^+$-Tree

```
p ← root
while p is not a leaf do
    choose child pointer P_i in p such that K_i ≤ v < K_{i+1}
    p ← P_i
end while
search leaf p for v
if v found then
    return pointer to the record
else
    return null
end if
```

Explanation

At each level, only one pointer is followed because the keys are sorted. Thus, the search path length equals the **height** of the tree, $O(\log_{n/2} N)$.

1.2 Range Query

A *range query* finds all records with keys in an interval $[v_1, v_2]$. This operation benefits from the leaf-level `next-leaf` pointers.

Algorithm 2 FindRange(v_1, v_2) – Range Query in a B$^+$-Tree

```
leaf ← FindLeaf(v1)                          ▷ Start from first relevant leaf
while leaf ≠ null and leaf.key ≤ v2 do
    output all records in [v1, v2] from current leaf
    leaf ← leaf.next
end while
```

Explanation

After locating the first leaf containing v_1, the query *sequentially scans* leaf nodes using `next-leaf` pointers until all keys $\leq v_2$ are covered. This makes range queries extremely efficient.

1.3 Performance Analysis

Performance Insights

- **Height:** $O(\log_{n/2} N)$.
- Example: For $N = 10^6$ records, block size = 4KB, $n \approx 200$. Tree height $\leq$ 3–4. $\Rightarrow$ Only 3–4 I/O operations per search.
- **Point Query:** follows a single path from root to leaf.
- **Range Query:** after one search, retrieves results with a linear scan across leaves.

2 Comparison with Binary Search Trees

Binary Search Tree (BST)	**B$^+$-Tree**
Height $\approx \log_2 N$	Height $\approx \log_{n/2} N$ (much smaller)
Each node stores 1 key	Each node stores up to hundreds of keys
Can become unbalanced (skewed tree)	Always balanced (all leaves at same level)
Search requires many node visits	Search requires few I/O operations
Designed for in-memory data	Optimized for disk-based / block storage
No leaf chaining	Leaves linked for efficient range queries